

Machine Learning Pipeline

A **Machine Learning (ML) pipeline** is a structured workflow that automates the end-to-end process of building, training, and deploying a model. Think of it as a **recipe** for turning raw data into predictions.

Typical Steps in an ML Pipeline

1. Data Collection

- Gather raw data from different sources: CSV, database, APIs, sensors.
- Example: Collecting bike-sharing trip history from Capital Bikeshare.

2. Data Preprocessing & Cleaning

(This is your focus – we'll dive deep next)

3. Feature Engineering / Feature Selection

- Transform raw data into meaningful features.
- Examples:
 - Extract **hour** or **day_of_week** from timestamp.
 - Encode categorical variables (One-Hot or Label Encoding).

4. Splitting the Dataset

- Divide data into **training**, **validation**, and **test** sets.
- Typical ratio: 70% train, 15% validation, 15% test.

5. Model Selection

- Choose ML algorithms suitable for your problem:
 - Classification → Decision Tree, Random Forest, Logistic Regression
 - Regression → Linear Regression, Random Forest Regressor

6. Model Training

- Fit the model on training data.

7. Model Evaluation

- Evaluate using metrics:
 - Classification → Accuracy, F1-score, ROC-AUC
 - Regression → RMSE, R^2 , MAE

8. Hyperparameter Tuning

- Improve performance using techniques like Grid Search or Random Search.

9. Model Deployment

- Make predictions on new, unseen data.
- Deploy via REST API, Web App (Streamlit, Flask, FastAPI), or production systems.

10. Monitoring & Maintenance

- Track model performance in real-world usage.
 - Retrain if performance drops.
-

Data Preprocessing

Data preprocessing is the **critical first step** in the pipeline. It ensures your raw data is **clean, consistent, and ready** for ML models.

Steps in Data Preprocessing

1. Handling Missing Values

- Identify missing or null values.
- Methods to handle:
 - Remove rows/columns → `df.dropna()`
 - Fill with mean/median/mode → `df.fillna()`
 - Use prediction models for imputation

2. Handling Outliers

- Detect outliers using IQR or Z-score.
- Handle:
 - Remove outliers
 - Cap values to a range

3. Encoding Categorical Variables

- Machine learning algorithms require numerical input.
- Techniques:
 - **Label Encoding** → Converts categories to numbers
 - **One-Hot Encoding** → Creates binary columns for each category

4. Scaling / Normalization

- Standardize numeric values for better model performance.
- Techniques:
 - **Min-Max Scaling** → Scales values between 0–1

- **Standardization** → Mean 0, Std 1 (**StandardScaler**)

5. Feature Engineering

- Create new features from existing ones to improve model performance.
- Examples:
 - From **date** → extract **month**, **weekday**, **hour**
 - Combine **height** and **weight** → **BMI**

6. Feature Selection

- Remove irrelevant or redundant features.
- Techniques:
 - Correlation analysis
 - Recursive Feature Elimination (RFE)
 - Tree-based feature importance

7. Train-Test Split

- Split preprocessed data into train/test sets for model evaluation.